

Nombre: Christhopher Irving Melgar Gómez

Grupo: 9697

Asignatura: Programación de dispositivos móviles

Actividad: 1 del módulo 5

Asesor: Cristian Cardoso Arellano

Fecha: 28 de octubre del 2026

Introducción.

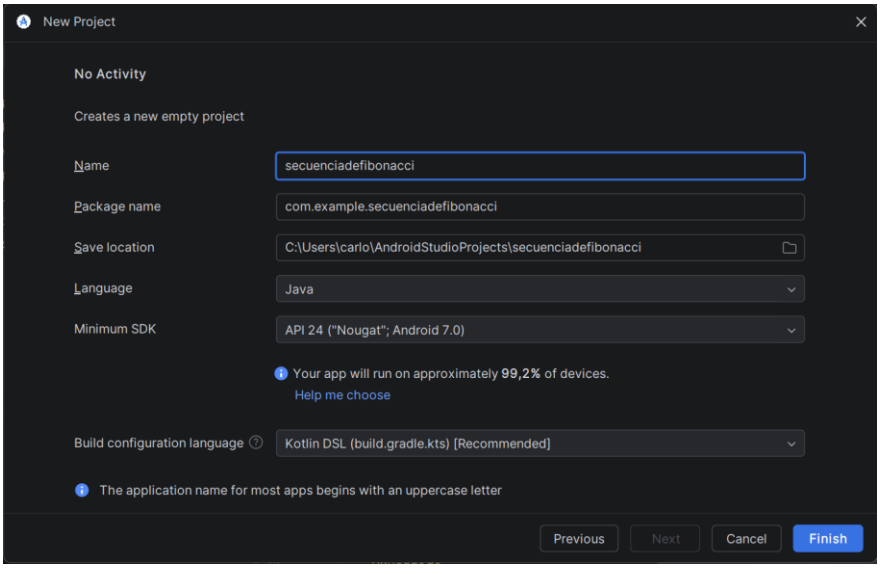
El objetivo de esta actividad en el módulo número 5 de la materia, es comprender los conceptos principales de las condicionales y sentencias que permitan generar una iteración y ciclos dentro de Nuestro programa.

El conocer estos conceptos nos permitirá generar aplicaciones dinámicas con diversos comportamientos acorde al usuario interactúe. El ciclo for así como el ciclo while son propiamente conceptos que permiten que una instrucción se repite un número indeterminado de veces mientras la condición del programa se cumpla.

Hoy estaré codificando uno de los programas que resultan clásicos en cualquier lenguaje de programación: La secuencia Fibbonaci. Esta es una secuencia iteractiva que permite conocer la progresión de un número acorde al anterior, y es empleado para conocer el ciclo for en cualquier lenguaje de programación acorde. La diferencia ahora en Android Studio es que desarrollaremos una interfaz gráfica mediante botones que nos permitan ver en tiempo real la progresión de los números y la interacción con el usuario.

Desarrollo de la actividad.

1) Lo primero es crear el Proyecto. Para ello crearé un empty activity con lenguaje Java bajo el nombre secuenciadefibonacci.



2) Lo Segundo es crear el documento XML que permita crear los botones para el ejercicio. Para ello en el Código se alcanza a percibir tanto la conexión con el programa (main activity), el vínculo de cada botón hacia el comportamiento del programa así como conceptos principales en XML para el diseño visual de las funcionalidades que le vamos agragando, como el alto, ancho, orientación, posición etc.

activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <!-- TextView que muestra el valor actual de la secuencia Fibonacci -->
    <TextView
        android:id="@+id/tvFibonacci"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="1"
        android:textSize="56sp"
        android:textStyle="bold"
        android:layout_marginBottom="40dp" />

    <!-- Botón de Avance (# Siguiente Clic) -->
    <Button
        android:id="@+id/btnAvance"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```xml
    <!-- Botón de Avance (# Siguiente Clic) -->
    <Button
        android:id="@+id/btnAvance"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Avance (Siguiente)"
        android:layout_marginBottom="16dp" />

    <!-- Botón de Retroceso (# Anterior Clic) -->
    <Button
        android:id="@+id/btnRetroceso"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Retroceso (Anterior)" />

</LinearLayout>
```

Explicación del Código: A partir de esta interfaz gráfica, estamos creando la parte con la que el usuario estará interactuando y es por ello que detallarla resulta una tarea fundamental para adaptarla a los dispositivos, que esté bien vinculada al main activity y además cumpla con un diseño agradable al usuario.

En la primera parte del Código, estamos detallando el ancho, alto, orientación, posición y el nivel de capa en el que se sitúa.

Después continúamos con el texto inicial en la secuencia de Fibonacci, y aparece un texto con las características visuales que nos permite mostrar el valor actual de la serie de Fibonacci. Este texto está vinculado con la parte lógica del programa y también con la interración de los botones conforme lo haga el usuario.

Posteriormente a ello, tenemos dos botones, los cuales vinculados a la parte lógica del programa, los describimos como anterior y siguiente.

3) Lo siguiente es crear la parte lógica del programa, la cuál estará conformada por la declaración de la interfaz, el control de clics sobre el usuario, la vinculación con la parte gráfica en XML, la declaración del valor inicial de la serie que será 1, posteriormente los botones de avance y de retroceso, posteriormente empieza la parte del ciclo iterativo del programa con la condicional if guardando el resultado de cada cálculo momentaneamente con memoization.

Para cada avance de la serie se llamará a una función llamada actualizar pantalla.



```java
public class MainActivity extends AppCompatActivity {

    // interfaz
    2 usages
    private TextView tvFibonacci;
    2 usages
    private Button btnAvance, btnRetroceso;

    // Control de los clics
    5 usages
    private int pasoActual = 1;
    1 usage
    private final int MAX_PASOS = 20; // Límite de términos a calcular

    // (caché para almacenar resultados recursivos previos)
    3 usages
    private Map<Integer, Long> memo = new HashMap<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // XML con las variables de Java
```

```java
        // XML con las variables de Java
        tvFibonacci = findViewById(R.id.tvFibonacci);
        btnAvance = findViewById(R.id.btnAvance);
        btnRetroceso = findViewById(R.id.btnRetroceso);

        // valor inicial correspondiente al #1 Clic
        actualizarPantalla();

        // 3. evento del Botón de Avance
        btnAvance.setOnClickListener( View v -> {
            if (pasoActual < MAX_PASOS) {
                pasoActual++; // Incrementa el número de clic
                actualizarPantalla();
            } else {
                Toast.makeText( context: this,  text: "Has alcanzado el límite máximo de la secuencia", Toast.LENGTH_SHO
            }
        });

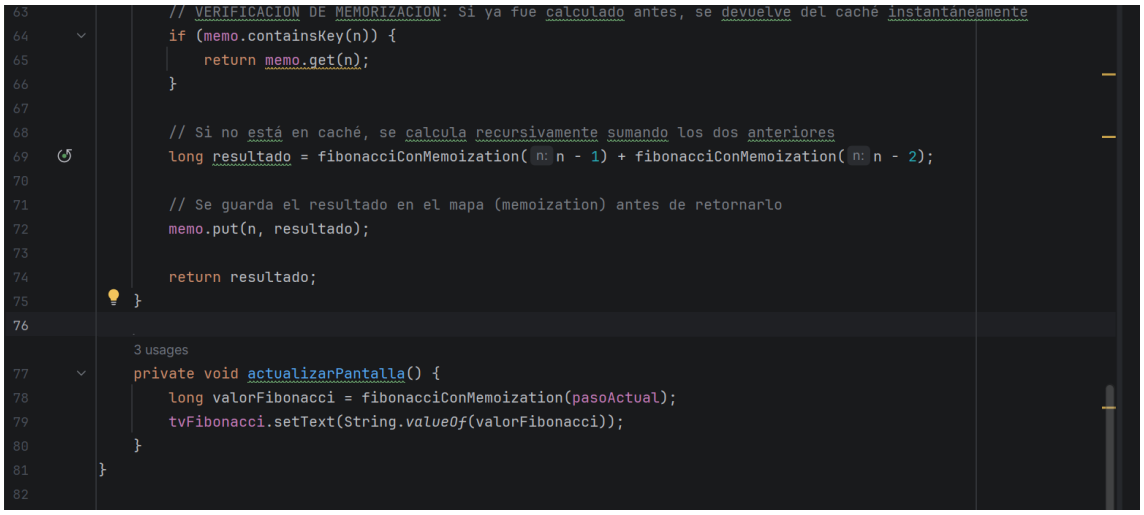
```

```java
        // 4. evento del Botón de Retroceso
        btnRetroceso.setOnClickListener( View v -> {
            if (pasoActual > 1) {
                pasoActual--; // Disminuye el número de clic
                actualizarPantalla();
            } else {
                Toast.makeText( context: this,  text: "Estás en el primer clic de la secuencia", Toast.LENGTH_SHORT).sho
            }
        });
    }

    3 usages
    private long fibonacciConMemoization(int n) {

        if (n <= 0) return 0;
        if (n == 1 || n == 2) return 1;

        // VERIFICACIÓN DE MEMORIZACIÓN: Si ya fue calculado antes, se devuelve del caché instantáneamente
        if (memo.containsKey(n)) {
```



```
        // VERIFICACION DE MEMORIZACION: Si ya fue calculado antes, se devuelve del caché instantaneamente
        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        // Si no está en caché, se calcula recursivamente sumando los dos anteriores
        long resultado = fibonacciConMemoization( n: n - 1) + fibonacciConMemoization( n: n - 2);

        // Se guarda el resultado en el mapa (memoization) antes de retornarlo
        memo.put(n, resultado);

        return resultado;
    }

    3 usages
    private void actualizarPantalla() {
        long valorFibonacci = fibonacciConMemoization(pasoActual);
        tvFibonacci.setText(String.valueOf(valorFibonacci));
    }
}
```

Conclusiones.

Aprendí a implementar funciones recursivas optimizadas para calcular la serie de Fibonacci, entendiendo la importancia de utilizar una estructura en caché (Map) para almacenar resultados previos. Esto me permitió mejorar el rendimiento del programa y evitar procesamientos innecesarios al navegar entre los números.

Comprendí cómo aplicar estructuras condicionales (if-else) en Java para controlar la lógica de avance y retroceso, asegurando que la aplicación responda de manera precisa a cada clic del usuario sin salir de los límites establecidos.

Logré conectar correctamente los componentes visuales en formato XML (TextView y Button) con el código fuente en Java dentro de la carpeta del proyecto, afianzando los fundamentos del desarrollo de aplicaciones móviles y la gestión de eventos mediante lambdas.